

Section 1. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns.

Section 2. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, by saving more bits than it costs to store.

Section 3. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical Huffman tree.

Section 4. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression.

Section 5. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, by saving more bits than it costs to store.

Section 6. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, by saving more bits than it costs to store.

Section 7. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per
Section 8. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixe
Section 9. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into
Section 10. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accep
Section 11. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.
Section 12. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger

Section 13. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.
Section 14. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 15. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.
Section 16. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.
Section 17. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into a dictionary of frequent patterns.
Section 18. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 19. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting e
Section 20. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only w
Section 21. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting e
Section 22. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting e
Section 23. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks,
Section 24. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 25. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 26. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting entries only if they are frequent enough to be worth the overhead of maintaining them. The engine then audits the dictionary against actual counts, and codes the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 27. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 28. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 29. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis.

Section 30. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 31. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself
Section 32. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 33. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 34. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream
Section 35. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the
Section 36. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the

Section 37. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost
Section 38. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks,
Section 39. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File C
Section 40. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resu
Section 41. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-
Section 42. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse

Section 43. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with
Section 44. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with
Section 45. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks
Section 46. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only with
Section 47. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive
Section 48. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses

Section 49. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts.

Section 50. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts.

Section 51. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 52. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts.

Section 53. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts.

Section 54. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts.

Section 55. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 56. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 57. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 58. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 59. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses the resulting mixed symbol stream.

Section 60. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 61. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder.

Section 62. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder.

Section 63. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder.

Section 64. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder.

Section 65. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder.

Section 66. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost is less than a threshold. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder.

Section 67. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only w
Section 68. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical
Section 69. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tie
Section 70. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when th
Section 71. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression us
Section 72. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into

Section 73. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary

Section 74. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 75. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when th

Section 76. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent pat

Section 77. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of fr

Section 78. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a pe

Section 79. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 80. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan.

Section 81. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one of the Huffman coders.

Section 82. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the blocks.

Section 83. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 84. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan.

Section 85. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan.

Section 86. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan.

Section 87. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts.

Section 88. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one of the Huffman coders.

Section 89. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 90. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 91. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision En
Section 92. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary
Section 93. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one c
Section 94. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-
Section 95. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical
Section 96. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 97. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 98. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 99. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns.

Section 100. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 101. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns.

Section 102. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns.

Section 103. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 104. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 105. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 106. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 107. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 108. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression.

Section 109. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the
Section 110. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual
Section 111. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and codi
Section 112. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed
Section 113. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-ti
Section 114. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictiona

Section 115. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 116. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,
Section 117. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro
Section 118. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,
Section 119. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks
Section 120. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t

Section 121. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman coder.

Section 122. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only once.

Section 123. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only once.

Section 124. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only once.

Section 125. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman coder.

Section 126. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman coder.

Section 127. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the

Section 128. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Co

Section 129. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonic

Section 130. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier

Section 131. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing

Section 132. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,

Section 133. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks

Section 134. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t

Section 135. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro

Section 136. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting

Section 137. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro

Section 138. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine build

Section 139. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the res
Section 140. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger stru
Section 141. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 142. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 143. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 144. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream v

Section 145. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in
Section 146. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine build
Section 147. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 148. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression u
Section 149. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, audi
Section 150. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting

Section 151. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the res
Section 152. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse
Section 153. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, audi
Section 154. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual co
Section 155. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 156. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision E

Section 157. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 158. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro
Section 159. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision E
Section 160. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File
Section 161. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 162. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictiona

Section 163. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,
Section 164. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger stru
Section 165. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression u
Section 166. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual co
Section 167. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The eng
Section 168. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine build

Section 169. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary again

Section 170. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary again

Section 171. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary again

Section 172. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary again

Section 173. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary again

Section 174. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary again

Section 175. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding
 Section 176. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding
 Section 177. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding
 Section 178. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding
 Section 179. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding
 Section 180. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding

Section 181. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a dictionary of patterns, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical Huffman tree.

Section 182. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical Huffman tree.

Section 183. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical Huffman tree.

Section 184. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical Huffman tree.

Section 185. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 186. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.

Section 187. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 188. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 189. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 190. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 191. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 192. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 193. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,
Section 194. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual
Section 195. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary
Section 196. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed
Section 197. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 198. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro

Section 199. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its
Section 200. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 201. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 202. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using mult
Section 203. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 204. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level

Section 205. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 206. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 207. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only
Section 208. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression u
Section 209. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into large
Section 210. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, audi

Section 211. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbols.

Section 212. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbols.

Section 213. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbols.

Section 214. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbols.

Section 215. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbols.

Section 216. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbols.

Section 217. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple
Section 218. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structures
Section 219. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns
Section 220. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 221. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 222. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structures

Section 223. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream v
Section 224. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 225. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 226. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing acce
Section 227. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictiona
Section 228. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Co

Section 229. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the
Section 230. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbols
Section 231. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 232. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary
Section 233. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbols
Section 234. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns

Section 235. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 236. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p
Section 237. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adap
Section 238. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 239. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level
Section 240. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine pro